

Reply to: J. Daniel Garcia (josedaniel.garcia@uc3m.es)

Support for contract based programming in C++

G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup

1. Introduction

This paper is based on P0380 ([P0380R1 \[1\]](#)). All changes are relative to latest working draft ([N4618 \[2\]](#)).

2. Summary of wording

2.1. A new attributes syntax

To simplify the way that contracts are specified, and in line with our previous design paper, we propose a new syntax that may be used for attributes.

We propose this new syntax to be usable by attributes taking a single argument, which is a valid expression.

```
[[contract-attribute: expression]]
```

where a *contract-attribute* is one of the following: **expects**, **ensures**, **assert**.

To avoid confusion, only the colon syntax for attributes is permitted in contract specifications

We also needed to support the idea of assertion levels needed by contracts. For this reason, we have introduced the concept of an attribute modifier that may appear after the attribute token itself.

```
[[contract-attribute modifier: expression]]
```

In this way we may specify contracts with assertion levels easily. We require that each attribute using the new proposed attribute syntax explicitly lists its accepted modifiers (if any).

Finally, we needed to introduce the ability to define the return value to be used in postconditions. We allow, that an attribute lists an identifier which is associated with the return value of a function.

```
[[contract-attribute modifier identifier: expression]]
```

With all this changes we can easily specify a contract:

```
int f(int x)
[[expects audit: x>0]]
[[ensures axiom res: res>1]];

void g() {
    int x = f(5);
    int y = f(12);
    //...
    [[assert: x+y>0]]
}
```

Note that while **assert(expression)** would expand as a function-like macro with the appropriate header, **assert:** is not a function-like invocation, so does not expand.

2.2. Functions versus function types

The current standard establishes a distinction between an attribute applied to a function and to the function type (the normative text uses the form "appertain to function type"). With that approach there would be a difference between the following cases:

```
[[attribute]] void f(); // Attribute applied to function
void f [[attribute]] (); // Attribute applied to function
void f() [[attribute]]; // Attribute applied to function type
```

However, we know of no example where an attribute is effectively used to annotate the function type. Besides, the current status makes it difficult to annotate functions with preconditions and postconditions. Consequently, in this paper we propose that attributes at the end of a function declaration apply to the function itself.

```
[[attribute]] void f(); // Attribute applied to function
void f [[attribute]] (); // Attribute applied to function
void f() [[attribute]]; // Attribute applied to function
```

Additionally, this solution solves the issue of being able to use attributes on lambda expressions (see Core issue 2097).

2.3 Contracts repetition

We require that any redeclaration of a function either has the contract or completely omits it.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int x); // OK. No contract.

int f(int x)
  [[expects: x>=0]]; // Error missing ensures and different expects condition

int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]]; //OK. Same contract.
```

Different argument names in redeclaration would be usually considered irrelevant.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int y)
  [[expects: y>0]]
  [[ensures z: z>0]]; // Should be OK
```

Consequently, we require that contracts are odr-identical in redeclarations, allowing for argument names variation.

2.4 Structured bindings and postconditions

One might imagine using structured bindings in postconditions:

```
std::tuple f()
  [[ensures [x,y]: x>0 && y.size()>0]];
```

However, we decided that this is something that might be considered for a future version. The same effect can be achieved currently as follows:

```
std::tuple f()
  [[ensures r: get<0>(r)>0 && get<1>(r).size()>0]];
```

2.5 Information of contract_violation

The information in `contract_violation` could be partially represented by a `source_location` object, from the Library Fundamentals V2 Technical Specification.

However, we defer this decision for a future version of this proposal.

2.6 Name lookup of contracts

Name lookup resolution in contracts may interact with the use of different build modes.

There are two answers here. The first one would be to make resolution dependent on the current build mode, requiring that only the contracts that would be evaluated in the current build mode need to parse correctly and pass the name lookup.

A second solution would be to make name lookup independent of build mode. In that case, all contracts would be required to pass name lookup independently of their assertion level. We have selected this second solution.

2.7 Identical contracts

We are also requiring in the wording that a redeclaration of a function that contains a contract needs to use the same contract (in the sense of ODR) that was present in the first declaration of the function.

The exact meaning would be to require contracts to be lexically the same token by token. This is a simpler definition, but would require that a redeclaration uses also the same names for functions arguments (if/when they are used in the contract).

A second solution would be to require the contracts to be logically equivalent which seems to introduce a number of additional implementation challenges.

A third option is to say that the functions must be textually equivalent except for a change of (parameter) variable names, but otherwise having the same structure. We have selected this option.

We have taken the route of requiring contract expressions to be odr-identical, except for the change of function parameter names.

2.8 Throwing violation handler

If a violation handler throws an exception, it is necessary to clarify what is the effect when the continuation mode is *off*.

One option could be that the exception propagates as the handler did not *return*. However, that design option would open the opportunity for continuation when the continuation has been set to *off*.

Another design alternative would be to unconditionally invoke `terminate()`.

2.9 Additional information in contract violation

Besides original information in **contract_violation**, a new member has been added to store the **assertion_level**. This value contains a string with the *assertion-level* of the contract that has been violated.

2.10 Invoking the handler

The proposal does not support the direct invocation of the violation handler. Allowing so, would imply access to handler supplied by the user, and could result into a security issue. Instead, we have added an additional *assertion-level* named **always**. Such assertion cannot be disabled and the check is performed even when the program is built with *build-level* set to *off*.

```
void f() {
    int x=get_value();
    [[assert always: x>0]]; // Invoke handler if x non-positive
    // ...
    if (x!=0) [[assert always: false]]; // Invoke handler if x!=0
    // ...
    [[assert always:true]]; // Unconditionally invoke handler
    // ...
}
```

This assertion level is available only for `[[assert]]`. A future extension might consider the usefulness of `always` for preconditions and postconditions.

Background of always level

The addition of the *always* assertion level was introduced during the Kona meeting. We reproduce here the relevant results from straw polls in the Evolution Working Group.

--	--	--	--	--	--

Question	SF	F	N	A	SA
Accept and proceed to LEWG	13	6	5	4	4
Accept and proceed to LEWG without <i>always</i>	5	13	9	5	2

Also, an up-down vote was taken:

Proposal as is	Proposal wihtout <i>always</i>
14	9

Note

This version of the document does not include wording for the *always* level.

2.11 Issues to be solved

Initialization of contract_violation objects

When a contract is broken, a **contract_violation** needs to be created with the corresponding information. There are several options on how such object should be populated with information.

Among the available options, one could be to leave this details as something to be defined by implementations. Otherwise, the exact population approach would need to be standardized.

Additional information in violation_info objects

Besides the information already defined in **contract_violation**, additional information might be useful. One example of such information is the assertion level of the contract assertion whose check generated the invocation of the violation handler.

3. Questions about contracts programming

What is the effect of violating a contract while evaluating the check for another expression

Nothing special needs to be considered. When the first contract is violated the corresponding action is taken.

```
bool positive(int * p) [[expects: p!=nullptr]] {
    return *p > 0;
}

bool g(int * p) [[expects: positive(p)]];

void test() {
    g(nullptr); // Contract violation when calling positive(nullptr)
}
```

Contract attribute as identifier

What happens if you try to use an identifier named as a contract attribute (e.g. **requires**, **audit**, ...)? Example:

```
X f(X & audit) [[ensures audit: audit.valid()]];
```

This is valid code. The first **audit** is interpreted as a *contract-level*. Then, the *conditional-expression* identifies the second **audit** correctly as the function argument.

4. Proposed Wording

Modify [5.1.5/6] as follows:

6 The closure type for a non-generic *lambda-expression* has a public inline function call operator (13.5.4) whose parameters and return type are described by the *lambda-expression's parameter-declaration-clause* and *trailing-return-type* respectively. For a generic lambda, the closure type has a public inline function call operator member

template (14.5.2) whose *template-parameter-list* consists of one invented type *template-parameter* for each occurrence of **auto** in the lambda's *parameter-declaration-clause*, in order of appearance. The invented type *template-parameter* is a parameter pack if the corresponding *parameter-declaration* declares a function parameter pack (8.3.5). The return type and function parameters of the function call operator template are derived from the *lambda-expression's* *trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of **auto** in the *decl-specifiers* of the *parameter-declaration-clause* with the name of the corresponding invented template-parameter. [Example:

```
auto glambda = [](auto a, auto&& b) { return a < b; };
bool b = glambda(3, 3.14); // OK
auto vglambda = [](auto printer) {
    return [](auto&& ... ts) { // OK: ts is a function parameter pack
        printer(std::forward<decltype(ts)>(ts)...);

        return [=]() {
            printer(ts ...);
        };
    };
};
auto p = vglambda( [](auto v1, auto v2, auto v3)
{ std::cout << v1 << v2 << v3; } );
auto q = p(1, 'a', 3.14); // OK: outputs 1a3.14
q(); // OK: outputs 1a3.14
```

— end example] This function call operator or operator template is declared **const** (9.2.2) if and only if the *lambda-expression's* *parameter-declaration-clause* is not followed by **mutable**. It is neither virtual nor declared **volatile**. Any *noexcept-specifier* specified on a *lambda-expression* applies to the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-declarator* appertains to the **type of the** corresponding function call operator or operator template. The function call operator or any given operator template specialization is a **constexpr** function if either the corresponding *lambda-expression's* *parameter-declaration-clause* is followed by **constexpr**, or it satisfies the requirements for a **constexpr** function (7.1.5). [Note: Names referenced in the *lambda-declarator* are looked up in the context in which the *lambda-expression* appears. — end note] [Example:

```
auto ID = [](auto a) { return a; };
static_assert(ID(3) == 3); // OK

struct NonLiteral {
    NonLiteral(int n) : n(n) { }
    int n;
};
static_assert(ID(NonLiteral{3}).n == 3); // ill-formed
```

— end example]

Modify [7/2] as follows:

2. A simple-declaration or nodeclspec-function-declaration of the form

*attribute-specifier-seq*_{opt} *decl-specifier-seq*_{opt} *init-declarator-list*_{opt} ;

is divided into three parts. Attributes are described in 7.6. *decl-specifiers*, the principal components of a *decl-specifier-seq*, are described in 7.1. declarators, the components of an *init-declarator-list*, are described in Clause 8. The *attribute-specifier-seq* appertains to each of the entities declared by the declarators of the *init-declarator-list*. [Note: In the declaration for an entity, attributes appertaining to that entity may appear at the start of the declaration **and**, after the *declarator-id* for that declaration, **or at the end of the function declaration**. — end note] [Example:

```
[[noreturn]] void f [[noreturn]] () [[noreturn]]; // OK
```

— end example]

In [7.6.1/1], modify the production for *attribute* as follows:

```
...
attribute-specifier:
    [[attribute-using-prefixopt attribute-list]]
    [[contract-attribute contract-levelopt identifieropt : conditional-expression]]
```

alignment-specifier

...

In [7.6.1/1], add a new production for *contract-attribute* as follows:

```
...
contract-attribute:
    expects
    ensures
    assert
...
```

In [7.6.1/1], add a new production for *contract-level* as follows:

```
...
contract-level:
    default
    audit
    axiom
...
```

Add a new section 7.6.11 Contracts attributes [**dcl.attr.contracts**]

1. A *precondition* is a predicate that should hold upon entry into a function. It expresses a function's expectation on its arguments and/or the state of objects that may be used by the function. Preconditions are expressed by *expects* attributes (7.6.10).
 2. A *postcondition* is a predicate that should hold upon exit from a function. It expresses the conditions that a function should ensure for the return value and/or the state of objects that may be used by the function. Postconditions are expressed by *ensures* attributes (7.6.11).
 3. An *assertion* is a predicate that should hold at its point in a function body. It expresses the conditions, on objects that accessible at its point in a body, that must be satisfied. Assertions are expressed by *assert* attributes (7.6.12).
 4. Preconditions, postconditions, and assertions are collectively called *contracts*. A contract shall have no observable effect in a correct program (a program where all contracts would be satisfied, if they were evaluated).
 5. Contract attributes are followed by a *conditional-expression*, which is a potentially evaluated expression (3.2).
- [Example]

```
void push(int x, queue & q)
[[expects: !q.full()]]
[[ensures: !q.empty()]]
{
    //...
    [[assert: q.is_valid()]];
    //...
}
```

— end example]

6. A contract attribute may be combined with the following *contract-levels*: *default*, *audit*, and *axiom* to express the *assertion-level* of the contract. When no modifier is provided, the *assertion-level* of the precondition is *default*.
[Note: A *default assertion-level* is expected to be used for those preconditions, postconditions and assertions where the cost of run-time checking is assumed to be small (or at least not expensive) compared to the cost of executing the function. An *audit assertion-level* is expected to be used for those preconditions, postconditions and assertions where the cost of run-time checking is assumed to be large (or at least significant) compared to the cost of executing the function. An *axiom assertion-level* is expected to be used for those preconditions, postconditions and assertions that are formal comments and are not evaluated at run-time. — end note]
7. A program with a contract expression that performs an observable modification of an object is ill-formed; no diagnostic required.

8. A contract of a `constexpr` function cannot refer to non-local objects that are not `constexpr`. [Example

```
int min=-42;
constexpr int max=42;

constexpr int f(int x)
    [[expects: min<=x]] // error
    [[expects: x<max]] // OK
{
    //...
}
```

— end example]

9. Every redeclaration of a function must contain exactly the same contract that was present in the first declaration of that function or completely omit the contract. The contracts expression shall be odr identical except for the change of function parameter names.

10. A translation may be performed with one of the following *build levels*: *off*, *default*, or *audit*. A translation with *build level* set to *off* shall not check any contract. A translation with *build level* set to *default* shall perform checking for *default* contracts. A translation with *build level* set to *audit* shall perform checking for *default* and *audit* contracts. [Note: The mechanism for selecting the *build level* is implementation defined. If no *build level* is selected, the *build level* is *default*. — end note] There shall be no programmatic way of setting, modifying or querying the *build level* of a translation unit.

11. If a function has multiple preconditions or postconditions that would be checked, their evaluation will be performed in the order they appear.

```
void f(int * p)
    [[expects: p!=nullptr]]
    [[expects: *p == 0]] // Only checked when p!=nullptr
{
    *p = 1;
}
```

12. A *violation handler* function is a function with the following signature:

```
void(const std::contract_violation &);
```

13. If a contract violation is detected, a *violation handler* will be invoked. It shall be implementation defined how the violation handler is established for a program and how the `std::contract_violation` argument value is set. There shall not be any programmatic way of setting or modifying the which violation handler is called in the event of a contract violation.

14. If a user-provided violation handler exits by throwing an exception and a contract is violated in a call to a function with a *non-throwing* exception specification, then the behavior is as if the exception was thrown within the function body. [Note: The function `std::terminate()` will be invoked. No stack unwinding will be performed. — end note] [Example:

```
void f(int x) noexcept [[expects: x>0]];

void g() {
    f(0); // std::terminate() if violation handler throws
    //...
}
```

— end example]

15. A translation may be performed with a *violation continuation mode*, which can be: *off* or *on*. A translation with a violation continuation mode set to *off* shall terminate execution by invoking `std::terminate()` after completing the execution of the violation handler. A translation with a violation continuation mode set to *on* shall continue execution after completing the execution of the violation handler. [Note: If no *continuation mode* is selected, the default *continuation mode* is *off*. — end note] [Note: A *continuation mode* set to *on* provides the opportunity to install a logging handler to instrument a pre-existing code base and fix errors before enforcing checks. — end note] [Example:

```
void f(int x)[[expects: x > 0]];
```

```
void g() {
    f(0); // std::terminate() after handler if continuation mode is off
        // Proceeds after handler if continuation mode is on
    //...
}

— end example]
```

Add a new section 7.6.11.1 Expects attribute [dcl.attr.contracts.expects]

1. The *contract-attribute* `expects` may be applied only to a function declaration.
2. The *contract-attribute* `expects` may appear multiple times applied to a function declaration with the same or different *assertion level* modifiers. [Example:

```
void f(object & o)
    [[expects: o.valid()]]
    [[expects audit: o.super_valid()]]
{
    //...
}
```

— end example]

3. The expression of a precondition from a function may use the function's arguments. The expression of a precondition from a function template or a member function of a class template may use the template arguments. The expression of a precondition from a public member function shall not use members from the protected or private interfaces. The expression of a precondition from a protected member function shall not use members from the private interface. The expression of a precondition from a *lambda-expression* may use any entity captured implicitly or explicitly. The expression of a precondition from a *lambda-expression* shall not use any entity that is not accessible by *lambda-expression*.

4. A function pointer shall not include preconditions. A call through a function pointer to functions with preconditions shall perform contract assertions checking once. [Example:

```
typedef int (*fpt)(int x) [[expects: x!=0]]; // Ill-formed

int g(int x) [[expects: x>=0]]
{
    return x+1;
}

int (*pf)(int) = g; // OK

— end example]
```

Add a new section 7.6.11.2 Ensures attribute [dcl.attr.contracts.ensures]

1. The *contract-attribute* `ensures` may be applied only to a function declaration.
2. The *contract-attribute* `ensures` may appear multiple times applied to a function declaration with the same or different *assertion level* modifiers. [Example

```
void f(object & o)
    [[ensures: o.valid()]]
    [[ensures audit: o.super_valid()]]
{
    //...
}
```

— end example]

3. The expression of a postcondition from a function may use the function's arguments. The expression of a postcondition from a function template or a member function of a class template may use the template arguments. The expression of a postcondition from a public member function shall not use members from the protected or

private interfaces. The expression of a postcondition from a protected member function shall not use members from the private interface. The expression of a postcondition from a *lambda-expression* may use any entity captured implicitly or explicitly. The expression of a postcondition from a *lambda-expression* shall not use any entity that is not accessible by *lambda-expression*.

4. A postcondition may introduce an identifier to represent the return value of the function. [Example:

```
int f(object & o)
    [[ensures res: res>0 && o.valid()]];

int g(object & o)
    [[ensures audit res: res!=0 && o.super_valid()]]
{
    //...
}
```

— end example]

5. A function pointer shall not include postconditions. A call through a function pointer to functions with postconditions shall perform contract assertions checking once. [Example:

```
typedef int (*fpt)() [[ensures r: r!=0]]; // Ill-formed

int g(int x)
    [[expects: x>=0]]
    [[ensures r: r>x]]
{
    return x+1;
}
```

```
int (*pf)(int) = g; // OK
```

— end example]

6. If a postcondition odr-uses in its expression an argument value and the function body modifies that value the program is ill-formed. [Example:

```
int f(int x)
    [[ensures r: r==x]]
{
    return ++x; // Ill-formed: Modified
}

int g(int * p)
    [[ensures r: p!=nullptr]]
{
    *p = 42; // OK. p is not modified
}
```

— end example]

Add a new section 7.6.11.3 Assert attribute [dcl.attr.contracts.assert]

1. The *contract-attribute* `assert` may be applied to a *statement*.
2. The expression of an assertion may use any object that can be accessed from the point of that assertion.

Modify clause [8.3.5/1]:

1. In a declaration τ **D** where **D** has the form

***D1** (parameter-declaration-clause) cv-qualifier-seq_{opt}*

ref-qualifier_{opt} noexcept-specifier_{opt} attribute-specifier-seq_{opt} and the type of the contained *declarator-id* in the declaration τ **D1** is “*derived-declarator-type-list*”, the type of the *declarator-id* in **D** is “*derived-declarator-type-list* **noexcept_{opt}** function of (*parameter-declaration-clause*) *cv-qualifier-seq_{opt}* *ref-qualifier_{opt}* returning τ ”, where the optional **noexcept** is present if and only if the exception specification (15.4) is non-throwing. The optional attribute-

specifier-seq appertains to the function **type**.

2. In a declaration $T\ D$ where D has the form

$D1\ (parameter-declaration-clause)\ cv-qualifier-seq_{opt}$
 $ref-qualifier_{opt}\ noexcept-specifier_{opt}\ attribute-specifier-seq_{opt}\ trailing-return-type$

and the type of the contained *declarator-id* in the declaration $\tau\ D1$ is “*derived-declarator-type-list* τ ”, τ shall be the *single type-specifier* **auto**. The type of the *declarator-id* in D is “*derived-declarator-type-list* **noexcept**_{opt} function of (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt} *ref-qualifier*_{opt} returning u ”, where u is the type specified by the *trailing-return-type*, and where the optional **noexcept** is present if and only if the exception specification is non-throwing. The optional *attribute-specifier-seq* appertains to the function **type**.

Add at the end of clause 10.3:

17. An overriding function shall have exactly the same contract that was declared for that function in the base class. However, the contract in the overriding function may be omitted in which case it has the same contract as was specified in the base class.

Modify clause 17.5.1.3/2:

2. A freestanding implementation has an implementation-defined set of headers. This set shall include at least the headers shown in Table 19.

Table 19 — C++ headers for freestanding implementations

Subclause	Header(s)
	><ciso646
18.2 Types	<cstdint>
18.3 Implementation properties	<cfloat> <limits> <climits>
18.4 Integer types	<stdint>
18.5 Start and termination	<stdlib>
18.6 Dynamic memory management	<new>
18.7 Type identification	<typeinfo>
18.8 Contract violation handling	<contract>
18.9 Exception handling	<exception>
18.10 Initializer lists	<initializer_list>
18.11 Other runtime support	<cstdalign> <cstdarg> <cstdbool>
20.15 Type traits	<type_traits>
29 Atomics	<atomic>

Modify clause 18.1/2:

2. The following subclauses describe common type definitions used throughout the library, characteristics of the predefined types, functions supporting start and termination of a C++ program, support for dynamic memory management, support for dynamic type identification, **support for contract violation handling**, support for exception processing, support for initializer lists, and other runtime support, as summarized in Table 32.

Table 32 — Language support library summary

Subclause	Header(s)
18.2 Common definitions	<cstdint>

		<cstdlib>
18.3	Implementation properties	<limits> <climits> <cfloat>
18.4	Integer types	<stdint>
18.5	Start and termination	<cstdlib>
18.6	Dynamic memory management	<new>
18.7	Type identification	<typeinfo>
18.8	Contract violation handling	<contract>
18.8.9	Exception handling	<exception>
18.8.9.10	Initializer lists	<initializer_list>
18.8.10.11	Other runtime support	<csignal> <csetjmp> <stdalign> <stdarg> <stdbool> <stdlib>

Add a new section after 18.8

18.8 Contract violation handling

1. The header `<contract>` defines a type for reporting information about contract violations generated by the implementation.

18.8.1 Header `<contract>` synopsis

```
namespace std {  
    class contract_violation;  
}
```

18.8.2 Class `contract_violation`

```
namespace std {  
    class contract_violation {  
    public:  
        int line_number() const noexcept;  
        const char * file_name() const noexcept;  
        const char * function_name() const noexcept;  
        const char * comment() const noexcept;  
        const char * contract_violation() const noexcept;  
    };  
}
```

1. The class `contract_violation` describes information about a contract violation generated by the implementation.

```
int line_number() const noexcept;
```

- 2. *Returns:* The line number where the contract violation happened.
- 3. *Remarks:* An implementation may return -1 if the location is unknown.

```
const char * file_name() const noexcept;
```

- 4. *Returns:* The source file name where the contract violation happened.
- 5. *Remarks:* An implementation may return `nullptr` if the location is unknown.

```
const char * function_name() const noexcept;
```

6. *Returns:* The name of the function where the contract violation happened.
7. *Remarks:* An implementation may return `nullptr` if the function is unknown.

`const char * comment() const noexcept;`
8. *Returns:* The text of the contract that was violated.

`const char * assertion_level() const noexcept;`
9. *Returns:* The text with the *assertion-level* of the contract that was violated.

Acknowledgements

Alexander Beels reviewed a previous version of this document. Andrzej Krzemienski and Melissa Mears provided comments to a draft of this version.

References

- [1] G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup. A contract design. [P0380R1](#). July, 2016.
- [2] Richard Smith. Working Draft, Standard for Programming Language C++. [N4618](#) November, 2016.